Answer:

1. Definition of Parallel Computing (1 Mark)

Parallel Computing is a computing technique in which multiple calculations or processes are executed simultaneously by dividing a large problem into smaller subtasks and executing them on multiple processors or CPU cores. This improves execution speed and overall system performance.

Parallel Programming is a computing technique in which **multiple operations are executed simultaneously** to solve a problem faster. A large task is divided into smaller subtasks, and these subtasks are executed at the same time using **multiple processors or CPU cores**.

---

2. Classification of Parallel Computers (4 Marks)

Parallel computers are classified using **two approaches**:

1. **Flynn's Taxonomy**
2. **Memory-Based Classification**

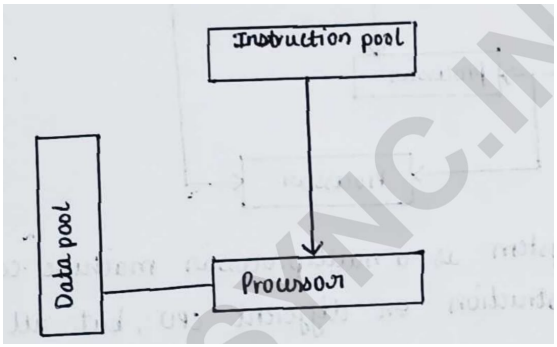
A) Flynn's Taxonomy

Definition:

Flynn's Taxonomy classifies parallel computers based on the number of **Instruction Streams** and **Data Streams** processed simultaneously.

Types of Flynn's Taxonomy

1. SISD (Single Instruction, Single Data)



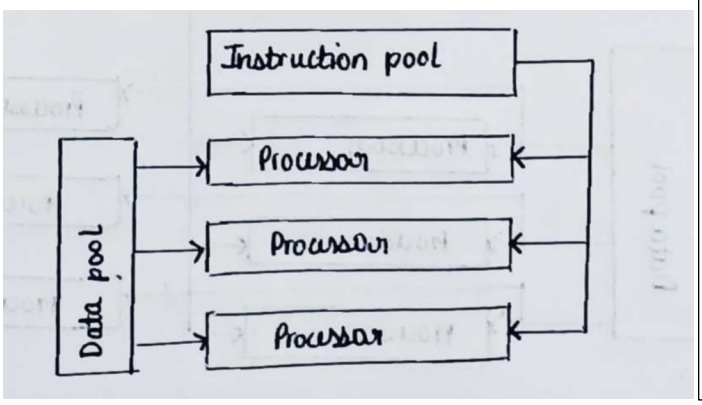
This diagram shows how a processor gets instructions and data to perform a task.
What each block means:
Instruction pool
Stores the instructions or commands to be executed.
Data pool
Stores the data on which the instructions operate.
Processor
Fetches instructions, gets the required data, and performs the computation.

- Executes **one instruction** on **one data item** at a time. The processor completes one operation before moving to the next.
- **Sequential execution.** Instructions are executed one after another in a fixed order.
- Based on **Von Neumann Architecture**. It uses a single processor to execute instructions and process data.
- **Used in traditional single-core computers.** A single processor handles the program's operations sequentially.

Example: Basic desktop computer with a single processor.

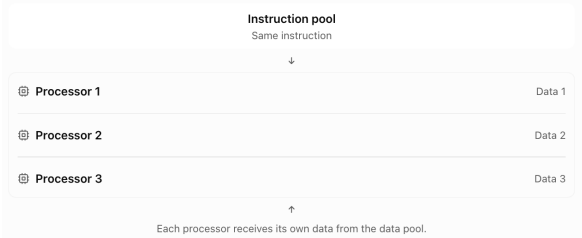
---

2. SIMD (Single Instruction, Multiple Data)



My previous explanation was for the basic processor diagram. This one is different because one instruction pool supplies the same instruction to multiple processors.

How to understand your diagram



- Executes **one instruction** simultaneously on **multiple data items**.
- A single control unit broadcasts the same instruction to multiple processing elements, which execute it on different data items.
- Suitable for **data parallelism**, where the same operation is performed on many data items.
- Highly efficient for large uniform datasets.

Applications

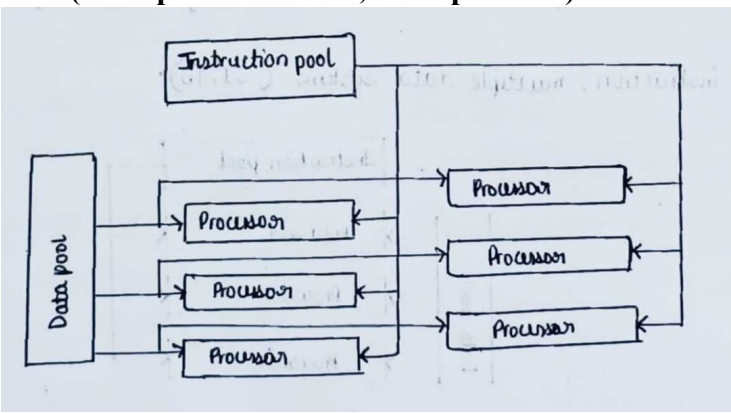
- Image processing
- Graphics processing
- Matrix operations
- Scientific computing

Example

Increasing the brightness of every pixel in an image.

---

3. MIMD (Multiple Instruction, Multiple Data)



- Executes **multiple instructions** on **multiple data items** simultaneously.
- Each processor executes its own program independently.
- Suitable for task parallelism, where different tasks are performed at the same time.
- Used in modern multi-core processors and distributed systems.

Applications

- Database servers
- Cloud computing
- Web servers
- Scientific simulations

Example

One processor runs a web server while another executes database queries.

B) Memory-Based Classification

1. Shared Memory System

- All processors share a **common memory**.
- Communication occurs through shared variables.
- Easy to program.
- May suffer from **memory contention**.

---

2. Distributed Memory System

- Each processor has its **own private memory**.
- Communication takes place through message passing over a network.
- Highly scalable.
- Used in clusters and supercomputers.

---

3. Comparison between SIMD and MIMD (5 Marks)

SIMD	MIMD
1. Full Form: SIMD stands for **Single Instruction, Multiple Data**.	MIMD stands for **Multiple Instruction, Multiple Data**.
2. Working: A single instruction is executed simultaneously on multiple data items.	Different processors execute different instructions on different data items independently.
3. Control Unit: SIMD uses a **single control unit (single instruction decoder)** to control all processing elements.	MIMD uses **multiple control units (multiple instruction decoders)**, allowing each processor to execute its own instructions.
4. Execution: SIMD performs **synchronous execution**, where all processors execute the same instruction at the same time.	MIMD performs **asynchronous (independent) execution**, where each processor executes its own instructions independently without waiting for other processors.
5. Parallelism: SIMD supports **data parallelism** because the same operation is applied to many data items.	MIMD supports **task parallelism** because different tasks are executed simultaneously.
6. Applications: Used in **image processing, graphics processing, matrix operations, and scientific computing**.	Used in **cloud computing, web servers, database systems, and multiprocessor computers**.
7. Example: Increasing the brightness of all pixels in an image.	One processor runs a browser while another downloads a file or executes a database query.
8. Cost & Complexity: SIMD is **less expensive and simpler** because all processors execute the same instruction.	MIMD is **more expensive and more complex** because each processor executes different instructions independently.

Example of SIMD

```
for(i=0; i<n; i++)
   C[i] = A[i] + B[i];
```

All array elements are added simultaneously using multiple processing elements.

Example of MIMD

- Core 1 → Runs a web browser
- Core 2 → Downloads a file
- Core 3 → Plays music
- Core 4 → Runs antivirus scan

Each processor performs a **different task** independently.

Conclusion

Flynn's Taxonomy classifies computers based on instruction and data streams. Among its types, **SIMD** is best suited for **data-parallel operations** such as image processing, while **MIMD** is suitable for **task-parallel applications** such as cloud computing, servers, and multi-core systems.